Université Alioune Diop de Bambey

L'excellence est ma constance, l'éthique ma vertu



UFR des Sciences Appliquées et TIC / Département TIC

Cours : Système d'exploitation

Monsieur Gaye

Table des matières

La partie matérielle et logicielle

Consigne : Ce document contient l'essentiel de la séquence

Chaque étudiant est invité à lire cette partie avant de commencer les activités

1 les constituants de l'ordinateur

- 1.1 La partie logiecielle
- 1.2 La partie matérielle

Rôles et fonctionnalités du SE

Dans cette partie les étudiants sont invités à revoir les notions de base du SE

- 1 Introduction
- 2 Types de systèmes d'exploitation
  - 2.1 Le multitâche coopératif
  - 2.2 Le multitâche préemptif
- 3 Place du système d'exploitation dans la machine
- 4 Fonction d'un système d'exploitation
  - 4.1 Gestion du processeur
  - 4.2 Gestion de la mémoire
  - 4.3 Gestion des entrées et sorties
  - 4.4 Gestion de la concurrence
  - 4.5 Gestion des objets externes (fichiers)
  - 4.6 Gestion de la protection
  - 4.7 Évolution historique des SE:
  - 4.8 Chargement du SE (Bootstrap ou amorçage)

Gestion du système de fichiers (SGF)

Dans ce cours les doivent s'appesantir sur la structure des fichiers, sur la gestion des blocs, les algorithmes de recherche des blocs

- 1 Introduction
- 2 Rôle du système de gestion de fichiers
- 3 Gestion des blocs libres
- 4 Système de fichier

La gestion de la mémoire et des entrées/sorties

- 1 Introduction
- 2 Mémoire sans va-et-vient ni pagination et Mémoire va-et-vient (swapping)
  - 2.1 Mémoire sans va-et-vient ni pagination
  - 2.2 Mémoire va-et-vient (swapping)

- 3 Gestion de l'allocation de la mémoire
- 4 Gestion de la pagination ou segmentation
- 5 Algorithmes de remplacement de pages

La gestion des processus et des ordonnancements

- 1 Introduction
- 2 Notion de processus
  - 2.1 Les états du processus
- 3 L'ordonnanceur
- 4 Gestion des interruptions
- 5 Synchronisation de processus
  - 5.1 Synchronisation de processus: exclusion mutuelle

La partie matérielle et logicielle

1 les constituants de l’ordinateur

Le mot informatique a été proposé en 1962 par Philippe Dreyfus du centre de calcul de Bull (est une société française spécialisée dans l'informatique professionnelle). C’est le traitement automatique de l'information par l'ordinateur.

Un système informatique est un ensemble de moyens logiciels et matériels nécessaires pour satisfaire les besoins informatiques des utilisateurs

1.1 La partie logicielle

Il est important de savoir qu'un logiciel est un programme.

Un programme décrit une procédure de calcul qui à partir d'informations en entrée (données) produit des informations en sortie (résultats).

On distingue deux types de logiciel :

- **Le système d'exploitation**

- **Le logiciel d'application**

Le système d'exploitation : un SE est un logiciel destiné à faire fonctionner l'ordinateur et qui permet de gérer le matériel et les autres logiciels et sert d'interface entre l'utilisateur et l'ordinateur.

Exemple : Windows 95, Windows 98, Windows 2000, Windows XP, UNIX.

Le logiciel d'application : un LA est un programme destiné aux tâches particulières et à chaque logiciel d'application correspondant une tâche précise.

Exemple : MS-Word : traitement de texte

MS-Excel : analyse financière et graphique

MS Power point : présentation assistée par ordinateur.

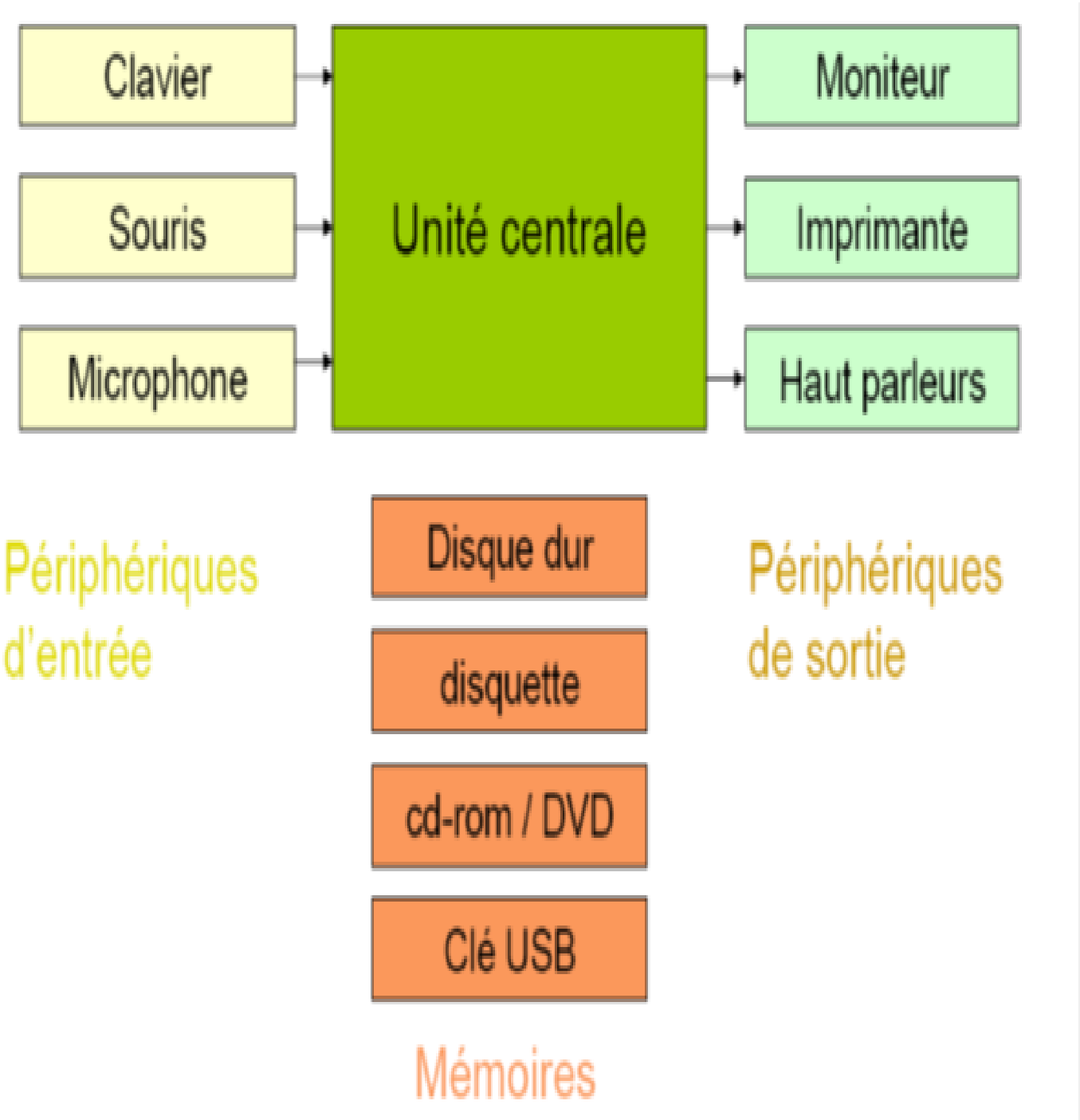
Les outils de développement et d'exécution d'application : un logiciel utilisé pour la programmation et le développement d'application comme VB, JAVA, C, Pascal

1.2 La partie matérielle

La partie matérielle est composée de :

L'unité centrale : c'est une boite qui comporte le processeur, la mémoire centrale et les mémoires de masse

Les périphériques : ceux sont les matériels que l'on connecte à l'unité centrale et qui servent à entrer ou sortir des informations.



Rôles et fonctionnalités du SE

1 Introduction

Le système d'exploitation est un logiciel qui permet d'assurer la liaison entre l'utilisateur, le matériel et l'application. Ainsi, lorsqu'un programme désire accéder à une ressource matérielle, il ne lui est pas nécessaire d'envoyer des informations spécifiques au périphérique, il lui suffit d'envoyer les informations au système d'exploitation, qui se charge de les transmettre au périphérique concerné via son pilote (drivers). En l'absence de pilotes il faudrait que chaque programme reconnaisse et prenne en compte la communication avec chaque type de périphérique.

Le noyau (kernel) : C'est la partie fondamentale d'un système d'exploitation, il est le gestionnaire de ressources de la machine, qui permet aux éléments matériels et logiciels de fonctionner ensemble. Pour ces raisons, il est

le premier logiciel chargé en mémoire (hors gestionnaire de boot)

2 Types de systèmes d’exploitation

Il n'existe pas de système d'exploitation efficace dans tous les contextes d'utilisation. On définit donc des familles de systèmes (mono-utilisateur, multiutilisateurs), en fonction des contraintes qui pèsent sur eux.

Mono-utilisateur

Ces systèmes d'exploitation n'acceptent qu'un seul utilisateur à un moment donné. Ils sont construits autour d'une machine virtuelle simple, et facilitent l'utilisation des différents périphériques. Ils peuvent être multitâches ou mono-tâches, mais n'implémentent pas les notions d'usager et de protection.

Qu'est-ce que le multitâche

Le multitâche est la capacité pour un système d'exploitation à gérer plusieurs programmes simultanément en leur attribuant tour à tour un pourcentage de temps processeur pour que ces programmes puissent s'exécuter. Nous distinguons deux types de multitâche :

Le multitâche coopératif

Le multitâche préemptif

2.1 Le multitâche coopératif

Dans cette configuration, chaque application qui s'exécute sur le système d'exploitation dispose du processeur puis le libère, permettant aux applications suivantes d'utiliser à leur tour le processeur. Le problème que nous rencontrons avec ce type de système est la dépendance des applications les unes par rapport aux autres. Si une application boucle sur elle-même, plus aucune application ne peut s'exécuter et nous allons avoir une défaillance du système d'exploitation. Nous trouvions ce genre d'implémentation dans les systèmes Windows 16 bits tel que Windows 3.1x ou Windows 95 avant l'OSR2.

2.2 Le multitâche préemptif

Dans cette configuration, chaque application dispose du processeur pendant un laps de temps déterminé à l'avance ou jusqu'à ce qu'une autre application ait une priorité supérieure à l'application en cours. L'ordonnancement (attribution d'un temps processeur pour une application) est fait par le système d'exploitation. Contrairement au multitâche coopératif, si une application se bloque, le système d'exploitation ne lui alloue plus de temps processeur et sera mise de côté. Le système restera donc stable.

3 Place du système d’exploitation dans la machine



Couche physique : Circuits électroniques

Langage machine : instructions servant à faire des calculs, des tests et à déplacer des données. Langage natif du processeur

Programmes et Procédures systèmes : Définissent la façon d'utiliser les ressources pour résoudre les problèmes informatiques des utilisateurs

Applications de haut niveau : Programmes utilisateurs

4 Fonction d'un système d'exploitation

Le SE fournit une gestion des ressources de la machine : processeurs, mémoires, disques, horloges, périphériques, communication interprocessus et inter-machines ; une base pour le développement et l'exécution de programmes d'application

4.1 Gestion du processeur

Le système doit gérer l'allocution du processeur aux différents programmes pouvant s'exécuter (processus). Cette allocution se fait par le biais d'un algorithme d’ordonnancement qui planifie l'exécution des programmes.

Typiquement, un processus est une instance d'un programme en cours d'exécution. Il est représenté au niveau du système d’exploitation par son code (texte), ses données, sa pile d'exécution, les valeurs courantes des registres

Un processus peut créer un autre processus. Il est susceptible d'être suspendu, démarré, arrêté

4.2 Gestion de la mémoire

Le système doit gérer l'allocution de la mémoire centrale entre les différents programmes pouvant s'exécuter (pagination/segmentation). Comme la mémoire physique est souvent petite pour contenir la totalité des programmes, la gestion de la mémoire se fait le principe de la mémoire virtuelle. A un instant donné, seules sont chargées en MC les parties de codes et données utiles à l'exécution.

Le mécanisme de mémoire virtuelle, ou swap, repose sur l'utilisation d'une mémoire de masse (type disque dur ou anciennement un tambour), afin entre autres, de permettre à des programmes de pouvoir s'exécuter dans un environnement matériel possédant moins de mémoire centrale que nécessaire (ou, vu autrement, de faire tourner plus de programmes que la mémoire centrale ne peut en contenir).

4.3 Gestion des entrées et sorties

Le système doit gérer l'accès aux périphériques c'est-à-dire l'utilisation entre les appels de haut niveau des programmes utilisateurs (ex getchar) et les opérations de bas niveau responsable de périphériques (ex clavier). C'est le pilote d'entrée sortie (driver) qui assure cette correspondance.

Les instructions de lecture et d'écriture sur les ports d'entrée/sortie du matériel ne sont permises qu'en mode noyau. Un programme d'application ne peut donc accéder directement au matériel sans passer par le système d'exploitation.

La différence entre mode noyau (privilégié) et mode utilisateur (non privilégié) est gérée directement au niveau du processeur. Elle n'a rien à voir avec la notion de super-utilisateur mise en œuvre sur certains systèmes

d'exploitation, qui est gérée au niveau logiciel dans le code du système d'exploitation. En fait, même le super-utilisateur d'un système passe le plus clair de son temps

4.4 Gestion de la concurrence

Comme plusieurs programmes coexistent en mémoire centrale ceux-ci peuvent communiquer pour échanger des données. Par ailleurs, il faut synchroniser l'accès aux données partagées afin de maintenir leur cohérence.

4.5 Gestion des objets externes (fichiers)

La mémoire centrale est une mémoire volatile. Aussi toutes les données devant être conservées au-delà de l'arrêt de la machine doivent être stockées sur une mémoire de masse (disque dur, CD rom, clé USB). La gestion de l'allocution de la mémoire de masse ainsi que l'accès aux données stockées sur la notion de fichiers et de système de gestion de fichiers (SGF).

Le système de fichiers fournit un modèle commode d'organisation des informations persistantes (c'est à dire dont la durée de vie est supérieure à celle des processus), avec une gestion indépendante du support matériel. Le plus souvent, l'organisation des fichiers se fait de façon arborescente. Le système de fichiers fournit les services

classiques, sur les fichiers : création, suppression, ouverture, fermeture, positionnement, lecture, écriture.

4.6 Gestion de la protection

Le système doit fournir des mécanismes garantissant ses ressources (CPU, mémoire, fichier) qui ne peuvent être utilisées que par des programmes auxquels les droits nécessaires ont été accordés. Il faut notamment protéger le système et la machine des programmes utilisateurs (mode d'exécution utilisateur et superviseur)

4.7 Évolution historique des SE :

Système de traitement par lots : Utilisation de cartes perforées pour les tâches (Traitement en lots (batch), travaux avec des besoins similaires, temps d'attente long, Processeur inactif). Introduction des disques (Ordonnancement des travaux, Utilisation du processeur de manière optimale). Multiprogrammation (Partitionnement de la mémoire en plusieurs parties). Spooling (Simultaneous Peripheral Operation On Line) (Utilisation d'une mémoire tampon, zone de la mémoire ou d'un disque)

Systèmes en temps partagé : Temps partagé ou multitâche (Extension logique de la multiprogrammation, Interaction de l'utilisateur et du système informatique,

Utilisation du clavier et de la souris, Utilisation de l'Ordonnancement d'exécution, Exécution interactive d'un processus(E/S), Commutation rapide du processeur vers le programme d'un autre utilisateur.

Systèmes des ordinateurs personnels : Apparition vers les années 70, Manque de fonctionnalité des processeurs, pas de protection du SE, ni multiutilisateurs ni multitâches, Optimisation de la convivialité et interactivité avec l'utilisateur SE MS-DOS-> plusieurs versions de Windows, Système multitâche OS/2 de IBM, SE du Mackintosh d'Apple, SE linux.

Systèmes parallèles : Systèmes multiprocesseurs (fortement couplés), Augmentation de la capacité de traitement, Economise de l'argent, Les processeurs partagent les périphériques, les stockages de masse et les alimentations, Augmentation de la fiabilité, Système tolérant aux pannes, Méthode de traitement (Multitraitements symétrique, Multitraitements asymétrique)

Systèmes temps réel : Système d'exploitation spécialisée, Possède des contraintes de temps fixes et bien définies, Employé comme dispositif de contrôle dans une application dédiée, Exemple de systèmes temps réel (Expériences scientifiques, imagerie médicale, contrôleurs d'équipements domestiques, systèmes d'armes…)

4.8 Chargement du SE (Bootstrap ou amorçage)

Introduction progressive du SE

_ **BIOS** (basic Input Output System) : Réside dans une mémoire de type ROM

_ **Pile CMOS** : Conserve les résultats du démarrage précédent

POST (power on self test): série de contrôle du matériel par le BIOS, vérification de proche en proche du bon état de fonctionnement du matériel

_ **MBR** (Master Boot Record) : Se trouve sur le premier secteur du disque avec 512 octets, Contient le SE,

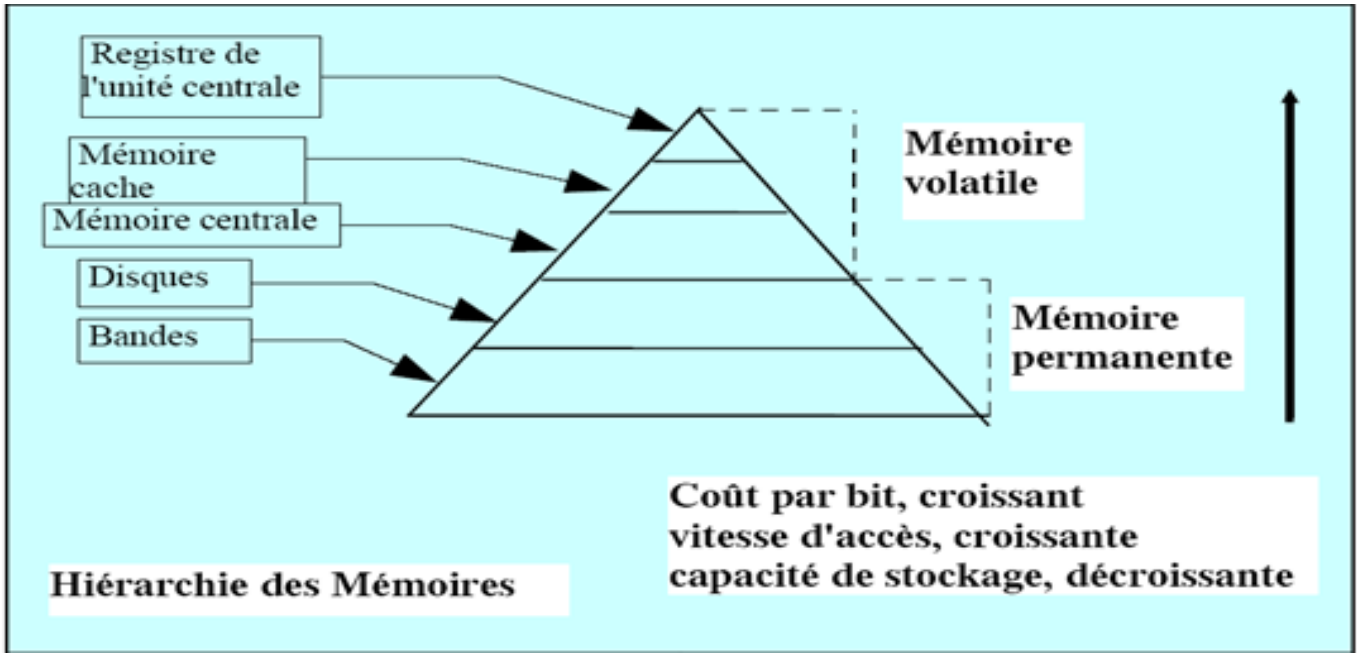


Gestion du système de fichiers (SGF)

Dans ce cours les doivent s'appesantir sur la structure des fichiers, sur la gestion des blocs, les algorithmes de recherche des blocs

1 Introduction

Le système d'exploitation doit pouvoir stocker à long terme de grandes quantités d'informations : ses propres modules, les programmes d'application, les bibliothèques de fonctions, les programmes et les données des utilisateurs.



La mémoire centrale est une mémoire volatile. Cependant, il faut stocker les données devant être conservées au-delà de l'arrêt de la machine sur un support de masse permanent. L'**unité** de conservation sur le support de masse est le **fichier**

Définition d’un fichier

Un fichier est une collection de données d’informations enregistrées de façon à être lues et traitées par ordinateur, et qui présente une entité pour l’utilisateur ou l’application. Un fichier peut être donc un programme source, un programme objet, des données, des résultats, un texte, une collection d’images, de sons, etc. Le **fichier** est l’unité logique de base du **S.G.F**

Un système de gestion de fichiers (**SGF**) est l'entité regroupant les fichiers mémorisés sur disque. Il contient les données des fichiers et un ensemble d'informations techniques.

Structuration d’un fichier

Selon les systèmes, différentes organisations sont proposées aux utilisateurs pour organiser les données dans les fichiers. Ceux-ci peuvent être organisés comme :
des suites d’octets : c’est l’organisation conceptuellement la plus simple. Le système de fichiers ne gère que des suites d’octets sans structure visible ; c’est leur interprétation par les différents programmes et le système (pour les fichiers considérés comme des exécutables) qui leur donne une signification. C’est l’organisation adoptée par de nombreux systèmes, comme les Unix, DOS,
des suites d’enregistrements : les fichiers sont structurés

en enregistrements de taille fixe, qui ne peuvent être lus et écrits qu'en totalité, sans possibilité d'insertion au milieu de la liste. C'était en particulier le cas de CP/M ;
un arbre d'enregistrements de taille variable : les fichiers sont organisés en enregistrements de taille variable, indexés chacun par une clé, et groupés par blocs de façon hiérarchique. L'ajout d'un nouvel enregistrement en une position quelconque peut provoquer un éclatement d'un bloc en sous- blocs, tout comme la suppression d'un enregistrement peut provoquer la fusion de blocs peu remplis. Cette organisation, de type fichier indexé est proposée par le système de fichiers ISAM **Q ;** *Indexed Sequential Access*

Type de fichier

Dans tous les systèmes de fichiers, la plupart des informations de structure sont elles aussi considérées comme des fichiers (spéciaux), de même que certains moyens de communication inter-processus (pipes nommées ou non, sockets).

Les types de fichiers les plus couramment définis sont les suivants :

- **Fichiers ordinaires** : ils contiennent les données des utilisateurs ;

- **Répertoires (ou catalogues)** : structure du système de fichiers permettant d'indexer d'autres fichiers, de façon hiérarchique ;
- **Fichiers spéciaux** de type **caractère** : modélisent des périphériques d'entrée/sortie travaillant caractère par caractère, comme les terminaux (claviers et écrans), les imprimantes, les pipes, les sockets ;
- **Fichiers spéciaux** de type **bloc** : modélisent des périphériques d'entrée/ sortie travaillant par blocs, comme les disques.

Tous ces fichiers peuvent avoir des extensions **.exe**(exécutable), **.o**(objet), **.txt**(texte).

2 Rôle du système de gestion de fichiers

Le système de gestion de fichiers permet de stocker, de conserver et de gérer des données sur une mémoire non volatile. Il est un outil de manipulation des fichiers et de la structure d'arborescence des fichiers sur disque. Le stockage des données répond à quatre besoins principaux

- Stocker des données de taille et de type variable de façon pérenne
- Pouvoir les identifier, les organiser et les retrouver, et éventuellement gérer leurs attributs (droit d'accès, expiration de validité, etc..).

- Garantir leur intégrité (sécurité)
- Avoir des performances d’accès, de lecture, d’écriture et de recherche en adéquation avec les besoins d’un système interactif

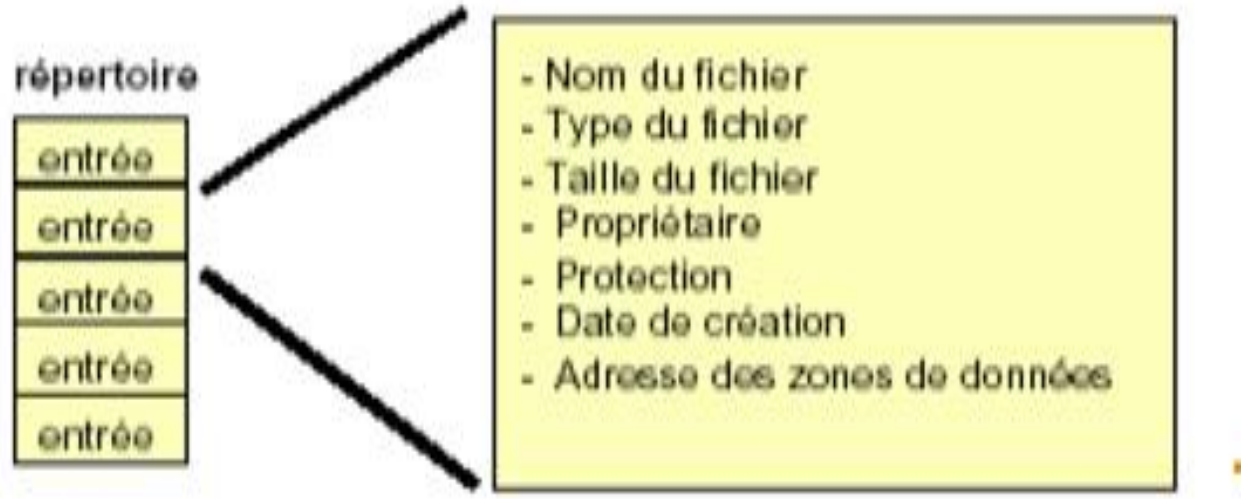
En revanche, l’utilisateur n’a généralement pas besoin de se préoccuper du type de stockage sous-jacent, de l’organisation des blocs, etc. Le SE permet un accès transparent au support pour l’utilisateur.

Structure d’un SGF

La structure du système de fichiers d’un SE est son niveau d’organisation le plus bas. Presque toutes les façons dont le SE interagit avec ses utilisateurs, ses applications et son modèle de sécurité dépendent de la façon dont il stocke ses fichiers dans son périphérique de stockage de base (généralement une unité de disque dur)

Pour conserver une trace des fichiers, le SGF possède généralement des répertoires (catalogues ou dossiers), qui sont eux-mêmes des fichiers spéciaux.

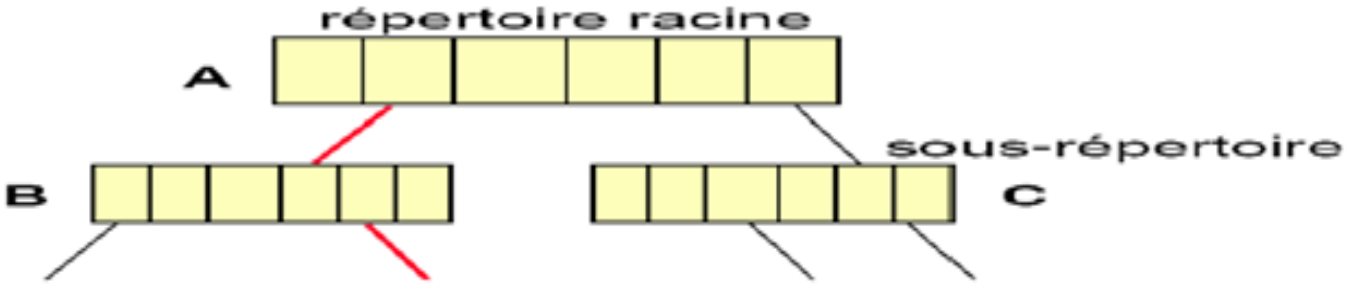
Composition d’un répertoire : Un répertoire est une zone disque réservée par le SGF. Le répertoire comprend un certain nombre d'entrées. Une entrée est allouée à chaque fichier du disque



- **Le modèle hiérarchique**

Les SE modernes adoptent une structure hiérarchique des fichiers. Chaque fichier appartient à un groupe d'autres fichiers. Chaque fichier appartient à un groupe d'ordre supérieur

On appelle ces groupes des **répertoires** ou **dossier** sous Windows. La relation entre fichiers et groupe est gérée par une table (**FAT** : **F**ile **A**llocution **T**able).



Fichiers physiques et fichiers logiques

Le système d'exploitation doit assurer la correspondance entre la représentation physique des informations et la représentation logique qu'en a l'utilisateur.

Le fichier physique est la représentation physique de l'information tandis que le fichier **logique** est un identificateur, qui désigne à la fois le fichier physique (entête ou i-nœud) et la représentation logique (nom), c'est-à-dire l'ensemble d'informations tel qu'il est vu par l'utilisateur.

Mode d’accès des fichiers

A chaque fichier correspond une liste de blocs contenant ses données. L'allocation est en générale non contigüe et les blocs sont donc répartis quasi-aléatoirement sur le disque. Les fichiers conservent l'ensemble de leurs blocs suivant deux méthodes :

Accès séquentiel : Les enregistrements du fichier ne peuvent être accédés que les uns à la suite des autres.

Accès direct (aléatoire) : Permettre un accès immédiat à n'importe quel enregistrement du fichier. L'accès à un enregistrement se fait en spécifiant sa position relative par rapport au début du fichier.

Accès indexé : on associe une clé à chaque enregistrement et l’on organise une table de correspondance (index) entre les clés et les numéros d’enregistrement. Une recherche sur l’index des clés permet de trouver l’adresse de l’enregistrement.

3 Gestion des blocs libres

Dès qu'on a choisi une taille de blocs (souvent 1/2 Ko à 2 Ko), on doit trouver un moyen de mémoriser les blocs libres. Les deux techniques les plus répandues sont les suivantes :

- table de bits : on gère une table comportant autant de bits que de blocs sur le disque. A chaque bloc du disque, correspond un bit dans la table, positionné à 1 si le bloc est occupé, à 0 si le bloc est libre (ou vice versa).

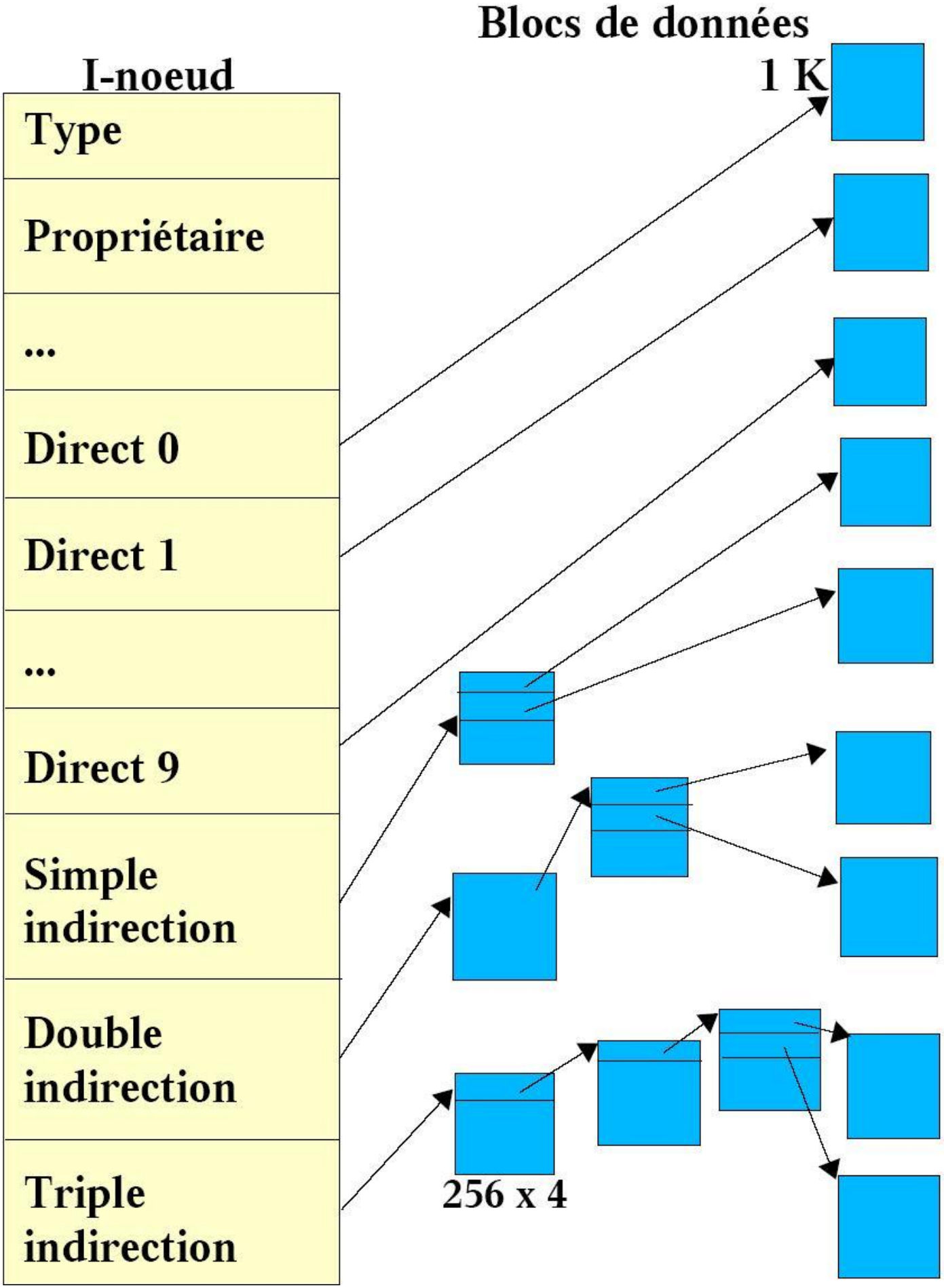
Dans la liste chainée : L'ensemble des blocs d'un fichier peut être chaînée sous la forme d'une liste. Chaque bloc contiendra des données ainsi que l'adresse du bloc suivant.



Cette méthode rend l'accès aléatoire aux éléments d'un fichier particulièrement inefficace lorsqu'elle est utilisée telle quelle. En effet **pour atteindre un élément sur le bloc *n* d'un fichier, le système devra parcourir les *n-1* blocs précédents**.

Pour l’index : Tout comme pour les listes chaînées, dans le cas de l'utilisation d'une table d'index (système UNIX) le noyau attribue de l'espace aux fichiers un bloc à la fois et

permet aux données d'être disséminées dans tout le système de fichiers. Pour maintenir une petite structure d'i-nœuds qui permet le stockage néanmoins des fichiers volumineux, la table des adresses des blocs disque se compose de 13 champs. Cette table est indépendante de la taille des fichiers. Les 10 premiers champs (direct) correspondent aux adresses des 10 premiers blocs de données sur disque. Pour les fichiers de plus de 10 blocs on a recours à des indirections. Le bloc n° 11 contient le numéro d'un bloc composé lui-même d'adresses de blocs de données. Un bloc (11) de 1024 octets pourra désigner jusqu'à 256 blocs de données, sachant que chacun d'eux est numéroté sur 4 octets. Le bloc n°12 contient une adresse à double indirection et le bloc n° 13 en contient une à triple indirection.


Blocs de données
I-noeud
1 K
Type
Propriétaire
...
Direct 0
Direct 1
...
Direct 9
Simple indirection
Double indirection
Triple indirection
256 x 4

4 Système de fichier

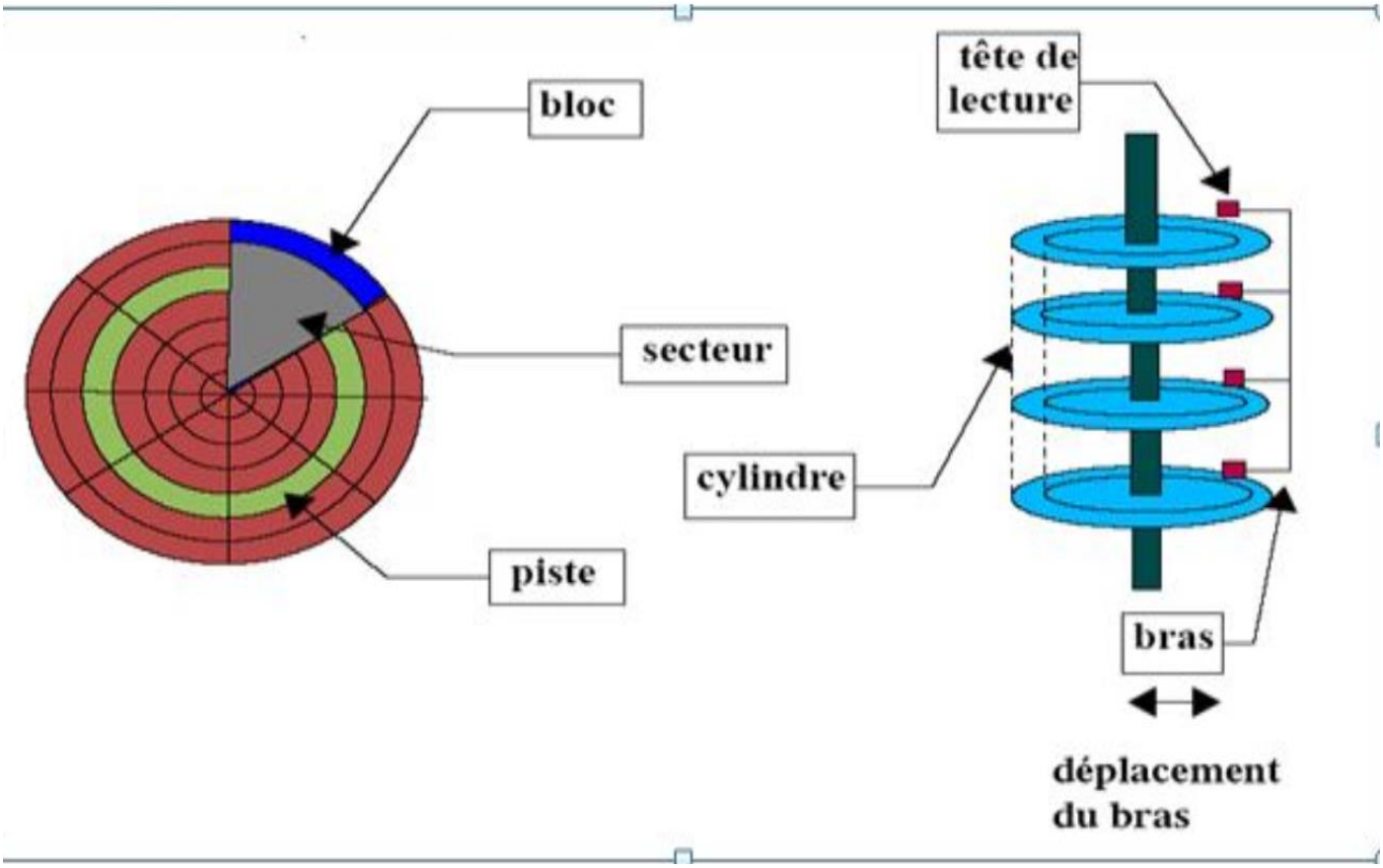
Un système de fichier consiste en une suite de blocs logiques, chacun contenant 512, 1024, 2048, ou n'importe quel multiple de 512 octets, selon la version du système. La taille d'un bloc To logique est homogène (fixe) dans un système de fichiers. Nous avons principalement deux systèmes de fichiers sous Windows :

FAT (File Allocation Table) et FAT32 ne permettent pas d'obtenir un niveau de sécurité sur les fichiers et répertoires. Le système de fichiers FAT ne supporte pas les partitions supérieures à 2 GO. FAT 32 est une évolution de FAT qui permet de dépasser cette limite. FAT 16 bits : 2 Go partition max, 8.3, FAT 32 bits : 8 Go partition max, 255 caractères.

NTFS (New Technologie File System) c'est le système recommandé pour Windows 2003 Server. Il accroit la sécurité en permettant de la contrôler au niveau des répertoires et des fichiers, gère la compression de disques ou de fichiers, permet de contrôler l'emploi des disques par les utilisateurs en appliquant des quotas ainsi que d'encrypter les données. Si vous prévoyez de configurer votre serveur en tant que contrôleur de domaine, il est impératif d'avoir au moins une partition formatée en NTFS pour stocker le volume système partagé. Partitions de 2

Structure du support physique : disque dur

Un fichier est découpé en blocs logiques auxquels sont attribués des blocs physiques du disque. Un disque est constitué de plateaux ; chaque plateau est divisé en pistes (cercles concentriques) et en quartiers d'angle fixe. Chaque intersection entre une piste et un quartier s'appelle un secteur.



Le SGF doit établir un lien entre le modèle logique des données, avec lequel travaillent l'utilisateur, et la réalité physique des dispositifs de stockage

Le formatage permet de définir la division des disques en pistes et secteurs. Généralement l'intersection d'un secteur avec une piste correspond au bloc, c'est-à-dire l'unité de transfert entre le périphérique et la mémoire centrale.

La gestion de la mémoire et des entrées/sorties

1 Introduction

Un programme a besoin de mémoire pour s'exécuter (stockage des instructions et des données)

Seules les instructions stockées en mémoire centrale peuvent être exécutées par le CPU

Le rôle du gestionnaire de la mémoire est de connaître les parties **libres** et **occupées**, d'allouer de la mémoire aux processus qui en ont besoin, de **récupérer** de la mémoire à la fin de l'exécution d'un processus et de traiter le recouvrement (va et vient) entre le disque et la mémoire centrale, lorsqu'elle ne peut pas contenir tous les processus actifs.

D'autre part, la taille d'un processus doit pouvoir dépasser la taille de la mémoire disponible, même si l'on enlève tous les autres processus. L'utilisation de pages **(mécanisme de pagination)** ou de segments **(mécanisme de segmentation)** permet au système de conserver en MC les parties utilisées des processus et de stocker, si nécessaire, le reste sur disque.

2 Mémoire sans va-et-vient ni pagination et Mémoire va-et-vient (swapping)

Dans il y a la notion de monoprogrammation et de multiprogrammation

2.1 Mémoire sans va-et-vient ni pagination

C'est le cas le plus simple (disons même rustique), dans lequel on considère que la mémoire physique effectivement disponible, que l'on accède directement au moyen d'adresses physiques (par opposition aux mécanismes d'adressage virtuel que l'on verra plus loin...).

Monoprogrammation : Dans ce cas, on n'a qu'un seul processus en mémoire à un instant donné, qui peut utiliser l'ensemble de la mémoire physique disponible. En pratique, une partie de l'espace d'adressage est dédiée au système d'exploitation, qui se compose des fonctions de démarrage et de gestion de bas niveau de la machine, situées en ROM et le système d'exploitation proprement dit. Le processus système (pour partie en RAM, pour partie en ROM ; la partie en ROM étant appelée BIOS [Basic Input Output System] et aussi les pilotes de périphériques.

2.2 Mémoire va-et-vient (swapping)

Multiprogrammation : La multiprogrammation est peu utilisée sur les petits systèmes dans ce contexte, mais l'est en revanche sur les gros systèmes dédiés au calcul intensif. Elle permet de diviser un programme en plusieurs processus et à plusieurs utilisateurs de travailler en temps partagé avec la même machine. Une solution pour optimiser la gestion de la mémoire est de la diviser en n partitions

Code translatable et protection : Avec le mécanisme de multiprogrammation, un processus peut être chargé n'importe où en MC. Il n'y a plus de concordance entre l'adresse dans le processus et l'adresse physique. Résolution de l'adressage par un registre particulier (registre de base)

@physique=@de base (registre) +@ relative (processus)

La technique de **swapping** permet de stocker temporairement sur disque des images de processus afin de libérer de la MC pour d'autres processus. La mémoire est divisée en partitions de taille variable. Pour un bon fonctionnement le système doit : interdire les E/S dans la zone soumise au swapping par le CPU, pouvoir activer un autre programme en cours d'exécution, réduire la durée du swapping qui est coûteuse

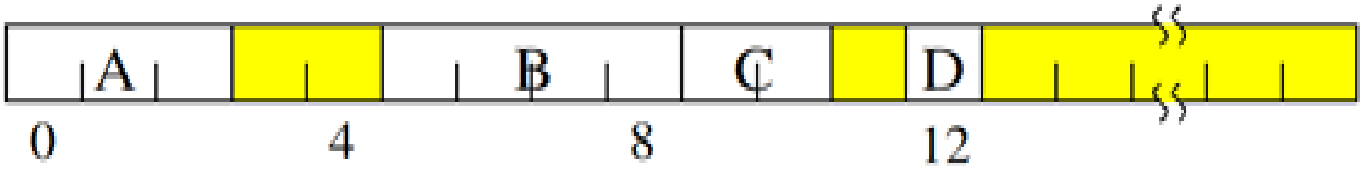
3 Gestion de l'allocation de la mémoire

Dans tous les cas, il faut disposer d'un mécanisme spécifique pour mémoriser les zones mémoire libres et occupées, minimiser l'espace perdu lors d'une allocation, et réduire autant que possible la fragmentation. Typiquement, ces mécanismes peuvent être de trois sortes.

Gestion par tables de bits : Avec cette technique, la mémoire est subdivisée en unités d'allocation, dont la taille peut varier de quelques mots à quelques kilo-octets. Chacune de ces unités est représentée par un bit dans une table d'allocation possédée par le système, celle-ci permettant alors de mémoriser l'occupation de la mémoire. On divise la MC en unités d'allocations de

quelques octets à quelques Ko. A chaque unité, correspond un bit de la table de bits : valeur 0 si l'unité est libre, **1** sinon. Cette table est stockée en MC. Plus la taille moyenne des unités est faible, plus la table occupe de place. Trois méthodes d'allocation sont utilisées dans la gestion avec une table de bit

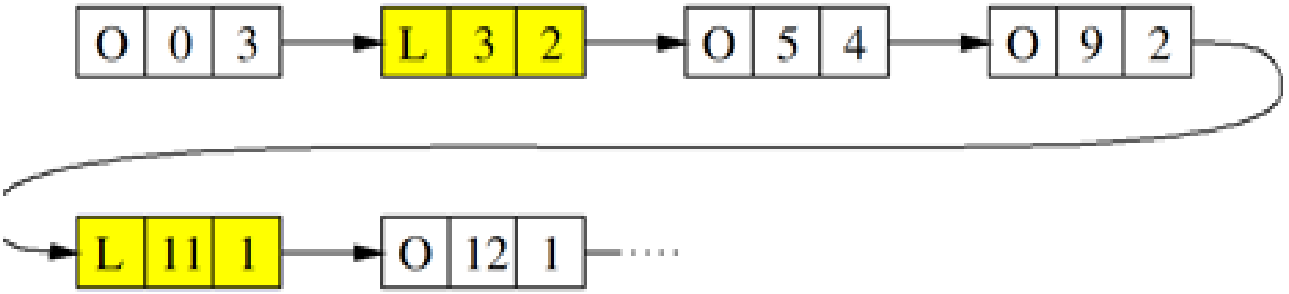
- **Best fit** : le bloc le plus proche de la taille requise
- **First fit :** le premier bloc qui satisfait la taille requise
- **Worst fit :** le plus grand bloc, de manière à avoir un bloc inutilisé le plus grand possible.



Cette approche présente en fait deux inconvénients. Le premier est le choix de la taille de l'unité d'allocation. Une taille trop petite imposera une table plus grande, et donc des temps d'opérations plus grands, alors qu'une taille trop grande causera des pertes de place plus importantes dans la dernière unité d'allocation, partiellement remplie. Le deuxième est la recherche de k bits libres dans la table, qui nécessite des manipulations coûteuses sur des masques de bits. Les tables de bits sont donc rarement utilisées.

Gestion par listes chaînées : Cette technique consiste à gérer une liste chaînée des segments libres et occupés, dans laquelle chaque segment est représenté par un bit d'état, son adresse de début, et éventuellement sa taille, et qui est triée par adresses de début croissantes. On utilise une liste chaînée des zones libres en MC. On applique : soit l'un

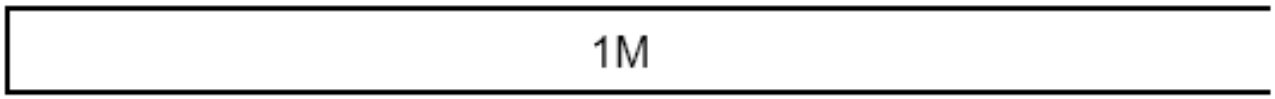
des **algorithmes précédents**, soit un algorithme de placement rapide **(quick fit) :** on crée des listes séparées pour chacune des tailles les plus courantes, et la recherche est considérablement accélérée.



A l'achèvement d'un processus ou à son transfert sur disque, il faut du temps (mise à jour des listes chaînées) pour examiner si un regroupement avec ses voisins est possible pour éviter une fragmentation excessive de la mémoire. En résumé, les listes chaînées sont une solution plus rapide que la précédente pour l'allocation, mais plus lente pour la libération.

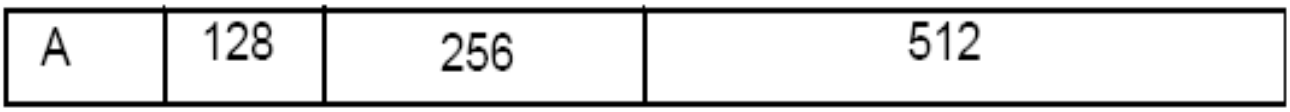
- **Gestion de la mémoire par subdivisions (ou frères siamois) :** Le gestionnaire mémorise une liste de blocs libres dont la taille est une puissance de 2 (1, 2, 4, 8 octets, ..., jusqu'à la taille maximale de la mémoire).

Par exemple, avec une mémoire de 1 Mo, on a ainsi 251 listes. Initialement, la mémoire est vide. Toutes les listes sont vides, sauf la liste 1 Mo qui pointe sur la zone libre de 1 Mo



Un processus A demande 70 Ko : la mémoire est fragmentée

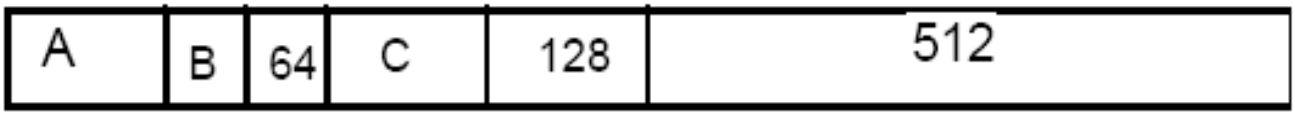
en deux compagnons (buddies) de 512 Ko ; l'un d'eux est fragmenté en deux blocs de 256 Ko ; l'un d'eux est fragmenté en deux blocs de 128 Ko et on loge A dans l'un d'eux, puisque 64 < 70 < 128 :



Un processus B demande 35 Ko : l'un des deux blocs de 128 Ko est fragmenté en deux de 64 Ko et on loge B dans l'un d'eux puisque 32 < 35 < 64 :



Un processus C demande 80 Ko : le bloc de 256 Ko est fragmenté en deux de 128 Ko et on loge C dans l'un d'eux puisque 64 < 80 < 128 :



4 Gestion de la pagination ou segmentation

La taille d'un processus doit pouvoir dépasser la taille de la mémoire physique disponible, même si l'on enlève tous les autres processus. Dans le processus de mémoire virtuelle : le SE conserve en mémoire centrale les parties utilisées des processus et stocke, si nécessaire, le reste sur disque. La mémoire virtuelle fait appel à deux mécanismes: **segmentation** ou **pagination**. La mémoire est divisée en **segments** ou **pages**. Sans recours à la mémoire virtuelle, un processus est entièrement chargé à

des adresses contiguës ; avec le recours à la mémoire virtuelle, un processus peut être chargé dans des pages ou des segments non contigus

La pagination

L'espace d'adressage d'un processus est divisé en petites unités de **taille fixe** appelées pages. La MC est elle aussi découpée en unités physiques de même taille appelées cadres

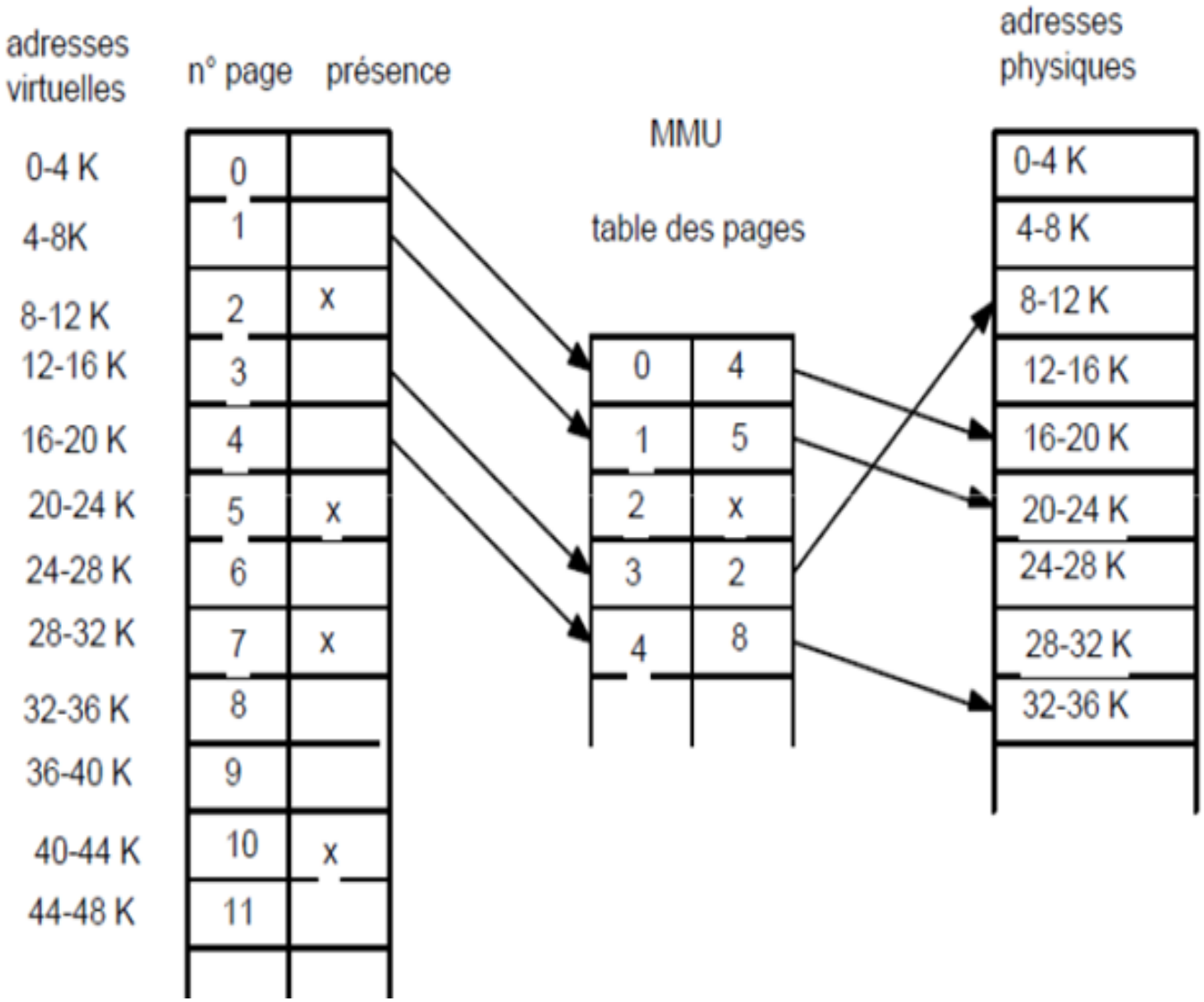
Pour un processus, le système ne chargera que les pages utilisées. Mais la demande de pages à charger peut-être plus élevée que le nombre de cadres disponibles. Une gestion de l'allocation des cadres libres est nécessaire.

Pour le calcul de l'adressage physique la **MMU** (Memory Management Unit ou unité de gestion de la mémoire) est chargé de traduire les adresses virtuelles en adresses physiques.

La MMU mémorise : les cadres physiques alloués à des processus (sous forme d'une table de bits de présence) et les cadres mémoire alloués à chaque page d'un processus (sous forme d'une table des pages). On dira qu'une page est mappée ou chargée si elle est physiquement présente en mémoire

Il y a **défaut de page** quand il y a un accès à une adresse virtuelle correspondant à une page non mappée. En cas de défaut de page, un **déroutement** se produit **(trap)** et le processeur est rendu au SE. Le système doit alors effectuer les opérations suivantes : déterminer la page à charger,

déterminer la page à décharger sur le disque pour libérer un cadre, lire sur le disque la page à charger, modifier la table de bits et la table de pages



La segmentation

L'espace d'adressage d'un processus est divisé en segments générés à la compilation

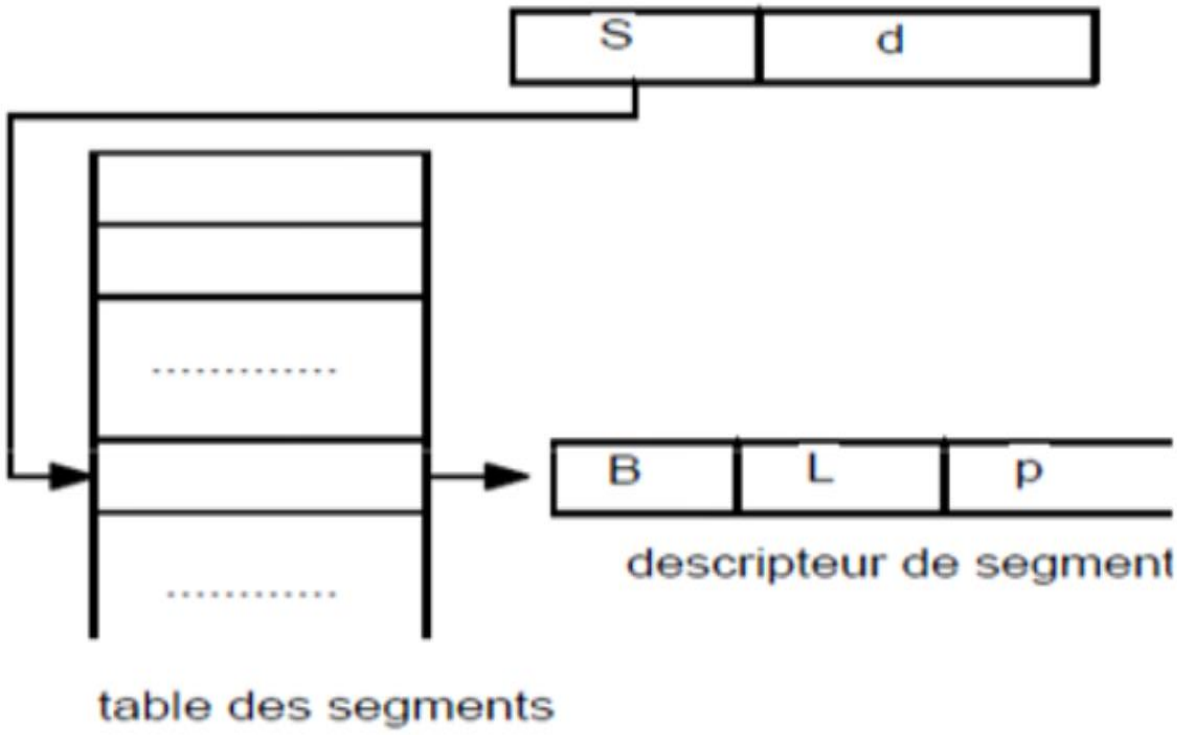
- *Un segment* est un ensemble d'adresses virtuelles contigües

- Chaque segment est repéré par son numéro S et sa longueur variable L.

- *L'adresse* est désignée par un couple <s, d> où s désigne ∈ le numéro du segment, et d'un déplacement dans segment d [0, L [

- L'adresse physique correspondant à l'adresse virtuelle (s, d) sera donc B+d ou B représente l'adresse de base (adresse physique de début de segment)

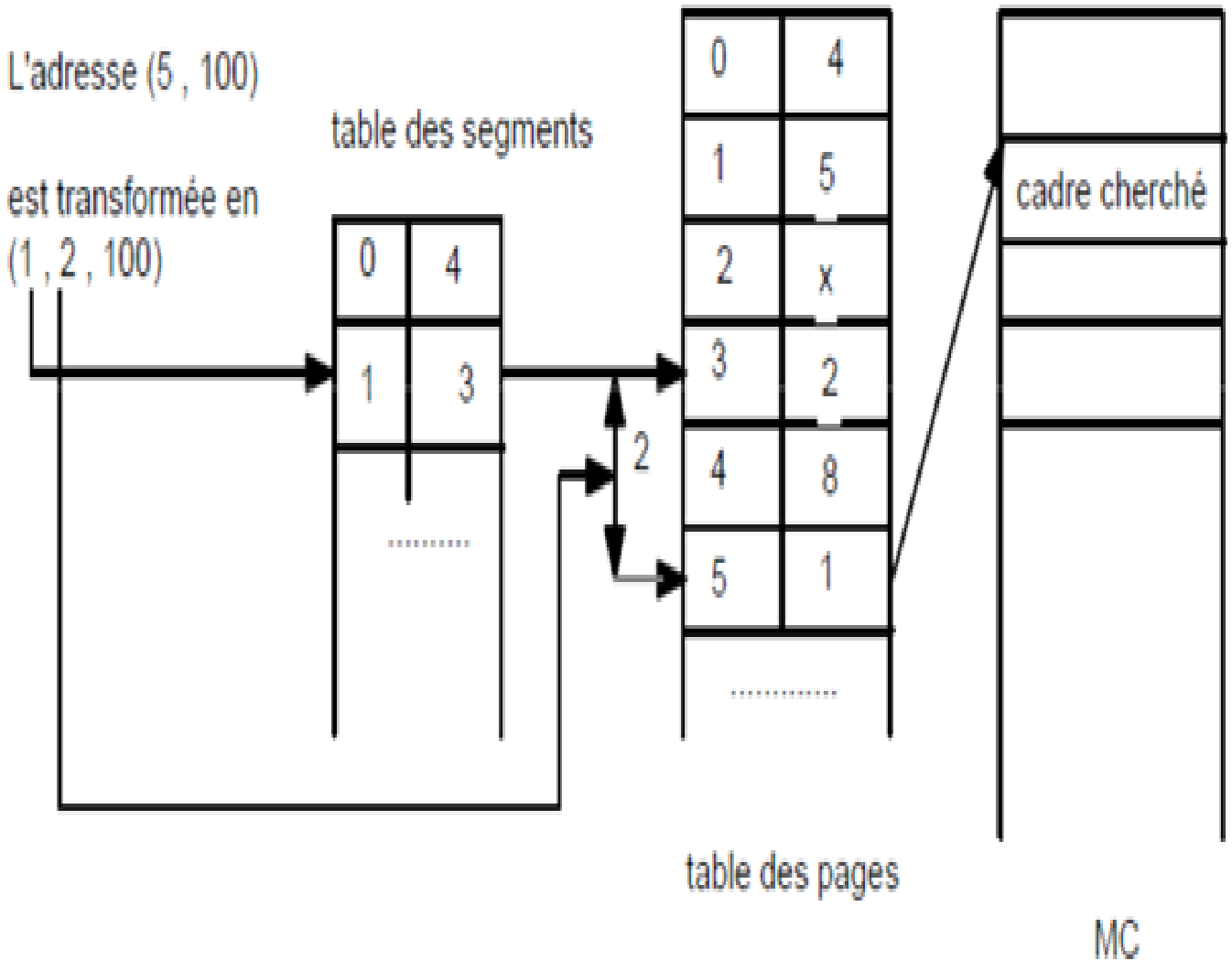
- La table des segments est utilisée pour calculer l'adresse physique



B : adresse de base (adresse physique de début du segment)
L : longueur du segment ou limite
p : protection du segment

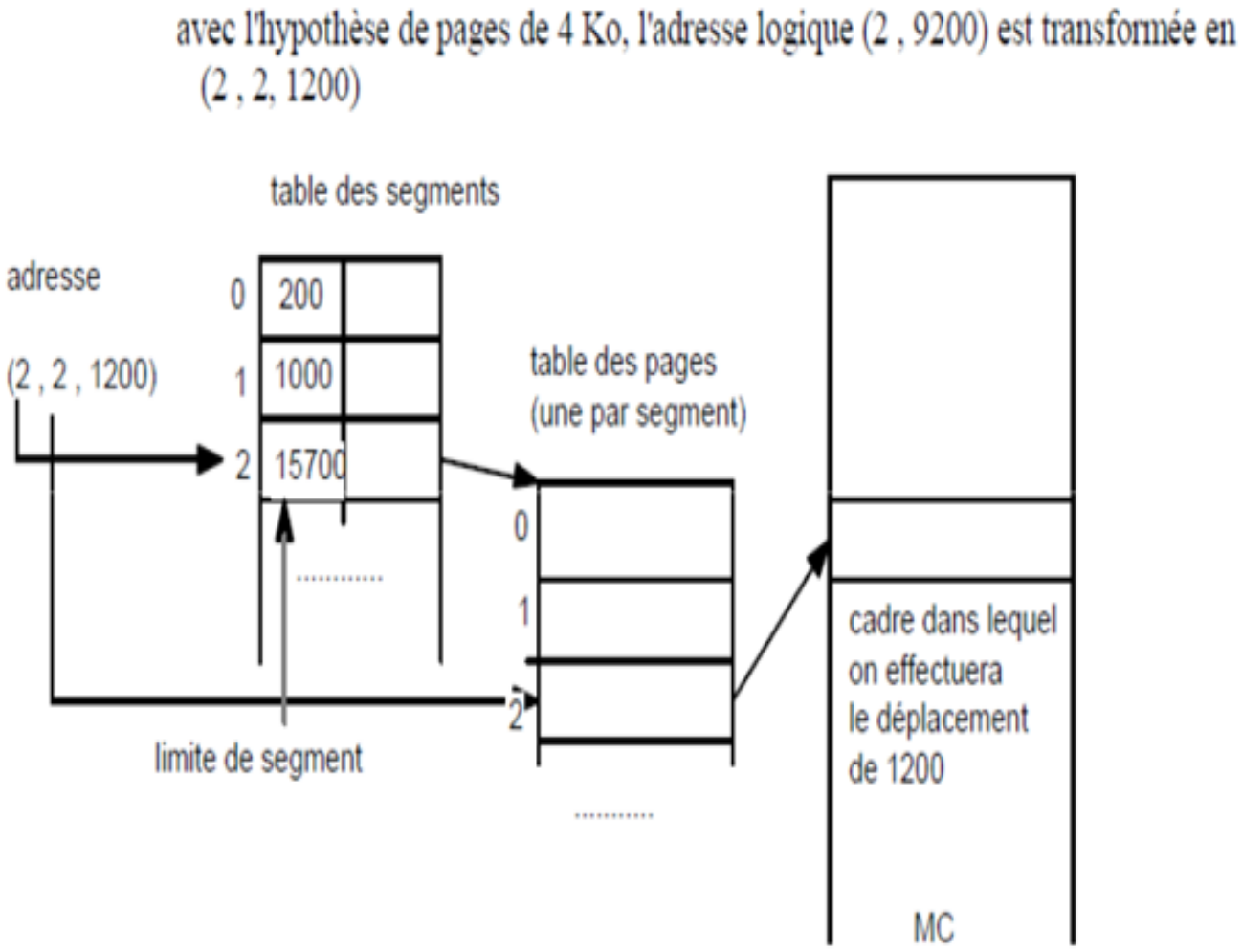
La pagination segmentée

Lorsque le système possède beaucoup de pages, la consultation de la table des pages peut être lente et inefficace s'il y a beaucoup de pages non chargées. On la segmente alors par processus. Chaque processus possède une table des segments de la table des pages qui le concernent. Une adresse (P, r), avec P n° logique de page et r déplacement dans la page, est transformée à la compilation en adresse (S, P', r), avec S n° d'un segment et P'.



La segmentation paginée

Lorsque la taille d'un segment s'avère importante, elle peut entrainer un temps de chargement long. D'où La solution de paginer les segments. Une adresse virtuelle (S , d), avec S n° de segment et d déplacement dans le segment, est transformée en (S , P , d'), où P est un n° de page et d' un déplacement dans la page



5 Algorithmes de remplacement de pages

Les méthodes reposent sur 3 stratégies :

- **une stratégie de chargement** qui choisit les pages à charger et l'instant de chargement

- une stratégie de placement qui choisit un cadre libre pour chaque page à charger

- une stratégie de remplacement destinée à libérer certains cadres. La recherche se fait soit sur l'ensemble des pages (recherche globale), soit sur les pages "appartenant" au processus (recherche locale). Les stratégies de placement sont dépendantes des stratégies de remplacement : on charge nécessairement une page dans le cadre qui vient d'être libéré.

On peut distinguer deux catégories de méthodes de chargement :

- pagination anticipée : on charge les pages à l'avance ; mais comment prévoir efficacement ?

- pagination à la demande : le chargement n'a lieu qu'en cas de défaut de page et on ne charge que la page manquante.

Les algorithmes de remplacement des pages :

Remplacement de page optimal : Le SE indexe chaque page par le nombre d'instructions qui seront exécutées avant qu'elle ne soit référencée. En cas de nécessité, le SE retire la page d'indice le plus élevé, c'est à dire la page qui sera référencée dans le futur le plus lointain.

Cet algorithme est pratiquement impossible à appliquer (comment calculer les indices des pages ?). Toutefois, avec un simulateur, on peut évaluer les performances de cet

algorithme et s'en servir comme référence pour les suivants. En outre, cet algorithme est stable.

NRU (not recently used) : Le SE référence chaque page par deux bits R (le plus à gauche) et M initialisés à 0. A chaque accès en lecture à une page, R est mis à 1. A chaque accès en écriture, M est mis à 1. A chaque interruption d'horloge, le SE remet R à 0. Les bits R-M forment le code d'un index de valeur 0 à 3 en base 10. En cas de défaut de page, on retire une page au hasard dans la catégorie non vide de plus petit index. On retire donc préférentiellement une page modifiée non référencée (index 1) qu'une page très utilisée en consultation (index 2). Cet algorithme est assez efficace.

FIFO (first in, first out) : Le SE indexe chaque page par sa date de chargement et constitue une liste chaînée, la première page de la liste étant la plus ancienne ment chargée et la dernière la plus récemment chargée. Le SE replacera en cas de nécessité la page en tête de la liste et chargera la nouvelle page en fin de liste.

LRU (least recently used) : En cas de nécessité, le SE retire la page la moins récemment référencée. Pour cela, il indexe chaque page par le temps écoulé depuis sa dernière référence et il constitue une liste chaînée des pages par ordre décroissant de temps depuis la dernière référence. L'algorithme est stable. Mais il nécessite une gestion coûteuse de la liste qui est modifiée à chaque accès à une page.

Variantes :

NFU (not frequently used) ou LFU (least frequently used) : le SE gère pour chaque page un compteur de nombre de références. Il cumule dans ce compteur le bit R juste avant chaque RAZ de R au top d'horloge. Il remplacera la page qui aura la plus petite valeur de Vieillissement (aging) ou NFU modifié : avant chaque RAZ de R, le SE décale le compteur de 1 bit à droite (division entière par 2) et additionne R au bit de poids fort. En cas de nécessité, on ôtera la page de compteur le plus petit. Mais en cas d'égalité des compteurs, on ne sait pas quelle est la page la moins récemment utilisée.

La gestion des processus et des ordonnancements

1 Introduction

Les premiers systèmes étaient monoprogrammes, c'est-à-dire qu'un seul programme réside en mémoire. La notion de processus est venue avec la multiprogrammation : créant ainsi des activités parallèles. Deux notions apparaissent avec la multiprogrammation : temps partagé, temps réel

En temps partagé : le nombre de processus est supérieur au nombre de CPU en activité : pseudo-parallélisme,

En temps réel : il y a autant de CPU que de processus : parallélisme réel

2 Notion de processus

Un processus est un programme qui est en cours d'exécution. On associe au processus un environnement processeur et un environnement mémoire appelés contexte du processus (on parle de ressources pour le processus).

Contexte d'un processus

Le contexte d'un processus également appelé bloc de contexte ou de contrôle est une structure de données qui décrit un processus en cours d'exécution. Ce bloc est créé au même moment que le processus et il est mis à jour en grande partie lors de l'interruption du processus afin de pouvoir reprendre l'exécution du processus ultérieurement.

Le contexte d'un processus comporte les informations suivantes :

- Le compteur ordinal : adresse de la prochaine instruction à exécuter par le processeur
- Les contenus des registres généraux : ils contiennent les résultats calculés par le processus
- Les registres qui décrivent l'espace qu'il occupe en mémoire centrale (l'adresse de début et de fin par exemple)
- Le registre variable d'état qui indique l'état du processus
- D'autres informations telles que la valeur de l'horloge, la priorité du processus

Commutation de contexte

L'opération qui consiste à sauvegarder le contexte d'un processus et à copier le contexte d'un autre processus dans l'unité centrale s'appelle changement ou commutation de contexte (context switching en anglais). La durée d'une commutation de contexte varie d'un constructeur à un

autre, elle reste toutefois très faible. Quand il y a changement de processus courant, il y a réalisation d'une commutation de mot d'état et d'un changement de contexte (multiprogrammation)

Tous ces registres forment ce que l'on appelle : un **mot d'état** (PSW, "Processus Status Word"). Ceci implique de savoir sauvegarder/restaurer un contexte d'exécution de programme. C'est pourquoi, tout processeur possède un contexte minimal d'exécution rassemblant les informations caractéristiques et suffisantes pour pouvoir, si elles sont rechargées dans les registres du processeur, reprendre l'exécution en son point de sauvegarde. Cet ensemble d'informations se présente sous la forme d'un « mot d'état programme » (**PSW** : Program Status Word), car la sauvegarde de l'état en mémoire centrale (en général par empilement) occupe un (double)-mot ;

Un processus est un programme qui est en cours d'exécution. On associe au processus un environnement processeur et un environnement mémoire appelés contexte du processus (on parle de ressources pour le processus).

2.1 Les états du processus

On appelle **ressource** toute entité dont a besoin un processus pour s'exécuter. Il en est ainsi du processeur physique, de la mémoire, des périphériques. Il en est également ainsi des données dont a besoin le processus et

qui seraient momentanément indisponibles. Il en est enfin ainsi de l'événement de fin d'exécution d'un processus

L’utilisation d'une ressource par un processus se fait en trois étapes :

- **Allocation**
-
- **Utilisation**
-
- **Restitution**

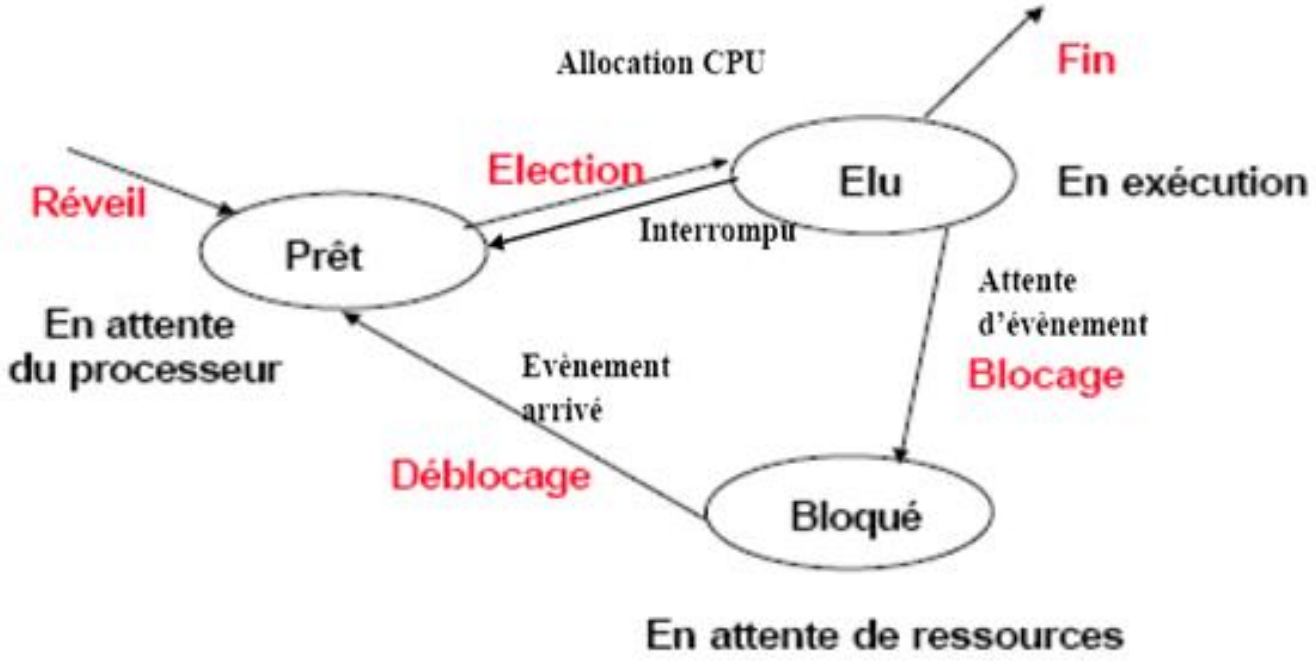
La phase d'**allocation** est la phase durant laquelle un processus demande à acquérir une ressource. Il **utilise** la ressource dès que l'allocation est réussie. Lorsqu'il n'a plus besoin de la ressource, le processus **restitue** la ressource.

Lors de son exécution, un processus est caractérisé par un état :

Elu : un processus est actif en mémoire centrale (en cours d'exécution).

Prêt : un processus est suspendu provisoirement en attente d'exécution, pour permettre l'exécution d'un autre processus.

Bloqué : un processus attend un évènement extérieur pour pouvoir continuer ; il est bloqué sur une ressource (par exemple au cours d'une lecture sur disque).



C'est donc le système d'exploitation qui détermine et modifie l'état d'un processus, sous l'effet d'évènements. Les évènements peuvent être internes au processus (une demande d'entrée/sortie peut faire passer de l'état en exécution ou ***élu*** à l'état ***bloqué***) ou extérieurs, provenant du système d'exploitation (une attribution de ressource fait passer de l'état ***bloqué*** à l'état ***prêt***).

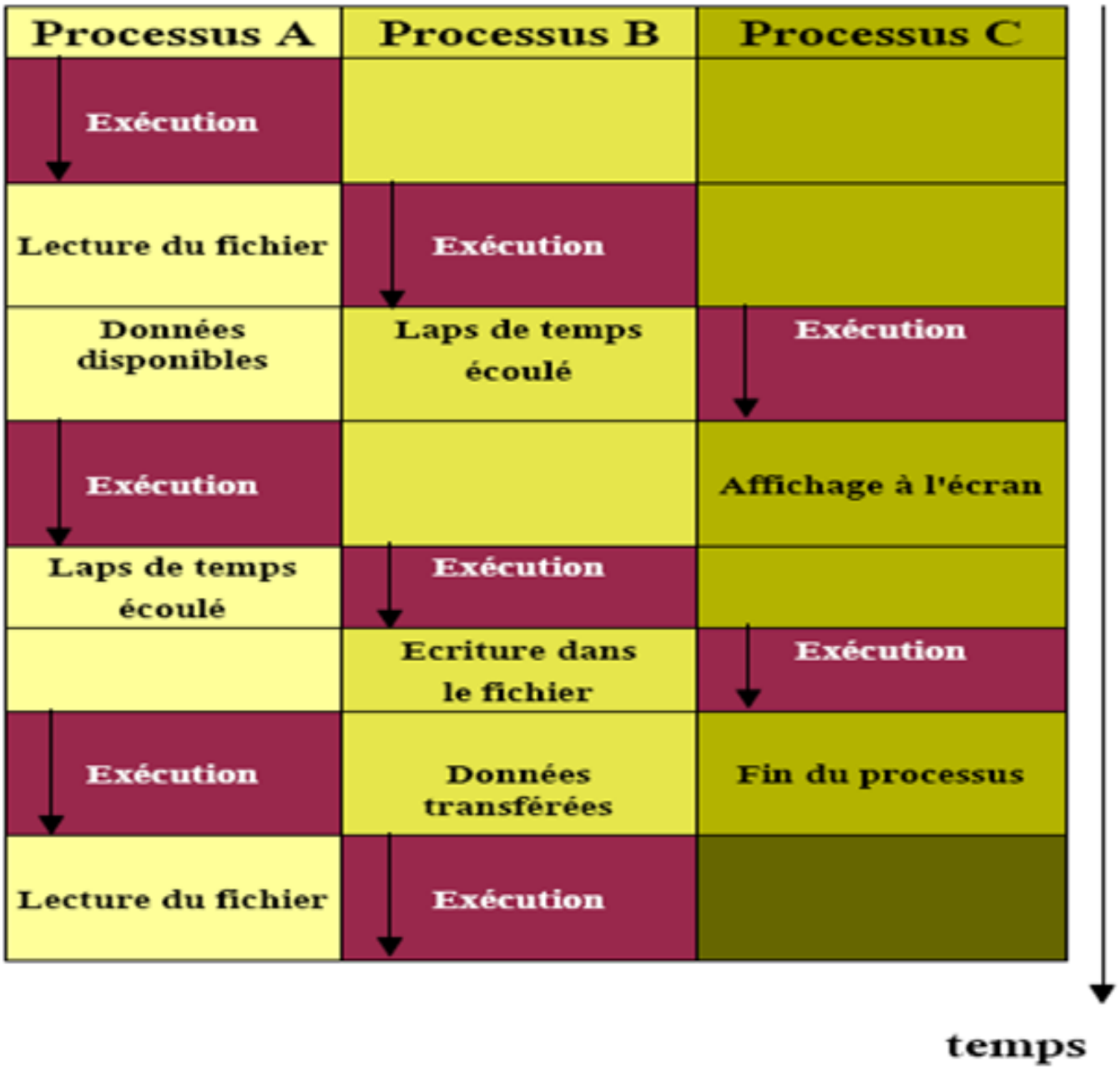
Un processus est toujours créé dans l'état prêt. Un processus se termine toujours à partir de l'état élu (sauf anomalie).

On appelle **ressource** toute entité dont a besoin un processus pour s'exécuter. Il en est ainsi du processeur physique, de la mémoire, des périphériques. Il en est également ainsi des données dont a besoin le processus et

qui seraient momentanément indisponibles. Il en est enfin ainsi de l'événement de fin d'exécution d'un processus

3 L'ordonnanceur

Objectif : Sur un intervalle de temps assez grand, faire progresser tous les processus, tout en ayant, à un instant donné, un seul processus actif (dans le processeur).



Rôle de l'ordonnanceur : Prendre en charge la commutation des tâches, qui assure le passage d'une tâche à une autre.

Pour construire un ordonnanceur, il suffit de disposer :

- **D’un mécanisme d'interruption**
- **D’un mécanisme de commutation de contexte**
- **Et d'une horloge.**

On choisit un **intervalle de temps** (quantum) et on génère une **interruption** à la fin de chaque quantum. Cette interruption provoque une commutation de contexte qui redonne le contrôle à l'ordonnanceur.

Un bon **algorithme** d'ordonnancement doit posséder les qualités suivantes :

- **Équitabilité** : chaque processus reçoit sa part du temps processeur
- **Efficacité** : le processeur doit travailler à 100 % du temps
- **Temps de réponse** : à minimiser en mode interactif
- **Temps d'exécution** : minimiser l'attente des travaux en traitement par lots (batch)
- **Rendement** : maximiser le nombre de travaux effectués par unité de temps

On distingue deux familles d'algorithme d'ordonnancement :

Algorithme sans réquisition (ASR) : le choix d'un nouveau processus ne se fait que sur blocage ou terminaison du processus courant (utilisé en batch par exemple).

On utilise :

- Soit l'ordonnancement dans l'ordre d'arrivée en gérant une file des processus. Cet algorithme est facile à implanter, mais il est loin d'optimiser le temps de traitement moyen
- Soit l'ordonnancement par ordre inverse du temps d'exécution (PCTE) : lorsque plusieurs travaux d'égale importance se trouvent dans une file, l'ordonnanceur élit le plus court d'abord.

Algorithme avec réquisition (AAR) : à intervalle régulier, l'ordonnanceur reprend la main et élit un nouveau processus actif

Un exemple d'algorithme avec réquisition le plus utilisé est celui de l'ordonnancement circulaire ou **tourniquet** (round robin)

- La méthode de tourniquet gère une liste circulaire de processus
- Concernant le mécanisme d'exécution utilisé, chaque processus dispose d'un quantum de temps pendant lequel il est autorisé à s'exécuter. Si le processus actif se bloque ou s'achève avant la fin de son quantum, le processeur est immédiatement alloué à un autre processus
- Si le quantum s'achève avant la fin du processus, le processeur est alloué au processus suivant dans la

liste et le processus précédent se trouve ainsi en queue de liste

L'algorithme du tourniquet (« round robin ») permet d'arbitrer les conflits entre les processus de même priorité, ou dont les phases de calcul sont trop longues.

4 Gestion des interruptions

Le traitement des interruptions par le système s'effectue en appelant une routine de traitement associée à chaque type d'interruption, dont l'adresse est stockée dans les cases d'un tableau indexé par le type d'interruption, appelé vecteur d'interruptions L2

Une interruption : C'est une commutation de mot d'état provoquée par un signal généré par le matériel. Ce signal est la conséquence d'un évènement qui peut être interne au processus et résultant de son exécution, ou bien extérieur et indépendant de cette exécution.

Un **vecteur d'interruption** correspond à plusieurs bits du mot d'état. C'est un emplacement mémoire contenant une adresse. L'arrivée de l'interruption provoque le branchement à cette adresse. A chaque cause d'interruption est associé un niveau d'interruption. Il existe trois principaux

- Interruptions externes (panne, opérateur, ...)

- Déroutement (erreur, division/0, débordement de tableau, ...),
- Appels au système SVC (E/S, ...).

numéro	*nature*	*traitant*
0	interruption d'horloge	*clockintr*
1	interruption disque	*diskintr*
2	interruption console	*ttyintr*
3	interruption d'un autre périphérique	*devintr*
4	appel au système (E/S)	*sottintr*
5	autre interruption	*otherintr*

S'il existe plusieurs niveaux d'interruption, on peut avoir à un moment donné, plusieurs indicateurs d'interruption positionnés en même temps. D'où l'utilisation de **priorités** sur les niveaux d'interruption. Si dans une interruption, une seconde se produit, ..., les processus ne progressent plus ; on fait des commutations de mots d'état en cascade. D'où le **masquage** et le **désarmement** de niveaux d'interruption.

En **masquant** un niveau d'interruption, on retarde la prise en compte des interruptions de ce niveau.

En **désarmant** un niveau d'interruption on annule l'effet des interruptions correspondantes. On peut réarmer un niveau désarmé.

5 Synchronisation de processus

Les processus peuvent partager des ressources communes (variables, fichiers, etc.)

Une ressource partageable peut être accessible par plusieurs processus à la fois ou par un seul à la fois ; dans ce dernier cas, on parle de **ressource critique (nombre de points d'accès = 1)**

Deux ou plusieurs processus peuvent coopérer pour élaborer un résultat en commun ; ils ont besoin de se communiquer des données. Comme les processus sont asynchrones (vitesses d'exécution différentes et non prévisible à cause des interruptions) il faut par moment les synchroniser.

La **synchronisation** se fait par communication de variables communes, de messages et de signaux

Les processus peuvent partager des ressources communes (variables, fichiers, etc.)

_ Une ressource partageable peut être accessible par plusieurs processus à la fois ou par un seul à la fois ; dans ce dernier cas, on parle de **ressource critique (nombre de points d'accès = 1)**

_ Deux ou plusieurs processus peuvent coopérer pour élaborer un résultat en commun ; ils ont besoin de se communiquer des données. Comme les processus sont asynchrones (vitesses d'exécution différentes et non

prévisible à cause des interruptions) il faut par moment les synchroniser.

_ La synchronisation se fait par communication de variables communes, de messages et de signaux

5.1 Synchronisation de processus : exclusion mutuelle

La solution conceptuellement naturelle à la résolution des problèmes d'accès concurrents consiste à interdire la modification de données partagées à plus d'un processus à la fois, c'est à dire définir un mécanisme d'exclusion mutuelle sur des portions spécifiques du code, appelées sections critiques.

On peut formaliser le comportement des sections critiques au moyen des quatre conditions suivantes :

1. Deux processus ne peuvent être simultanément dans la même section critique ;
2. Aucune hypothèse n'est faite sur les vitesses relatives des processus, ni sur le nombre de processeurs ;
3. Aucun processus suspendu en dehors d'une section critique ne peut bloquer les autres ;
4. Aucun processus ne doit attendre trop longtemps avant d'entrer en section critique.

Lorsque deux ou plusieurs processus sont en compétition pour le partage d'une ressource critique, on dit qu'ils

sont en exclusion mutuelle on les appelle des processus concurrents

Pour la ressource CPU, tous les processus sont en compétition. Dans ce cas, les conflits d'accès à cette ressource sont résolus par l'ordonnanceur

Les instructions d'un processus appartiennent soit à la section critique soit à la section non critique du processus.

Les instructions du processus qui permettent d'accéder à la ressource critique en lecture /écriture forment la section critique.

- On dit que les processus sont en exclusion mutuelle pour l'exécution de leur section critique

Les méthodes proposées pour résoudre le problème de l'exclusion sont classées en deux catégories :

- **Algorithme par attente active**
- **Algorithme par attente passive**

On considère que pour entrer dans sa section critique, le processus exécute une séquence d'instructions appelée **prologue** ; pour sortir la séquence d'instruction s'appelle **épilogue.** Si tous les processus ont les mêmes droits sur la ressource, ils exécutent le même prologue et le même épilogue

Modèle d'exécution d'un processus

Contexte commun : variables partagées par les processus

Corps d'un processus ***Pi***

Début

Répéter

première-partie-du-programme

Prologue

Section critique

Epilogue

reste-du-programme

Jusqu'à Faux

Fin

Règles à respecter

Règle 1 : Si un processus Pi s'exécute dans sa section critique, aucun autre processus ne doit pouvoir entrer dans sa section critique

Règle 2 : si aucun processus Pi n'est en section critique et qu'un processus veut entrer en section critique, seuls les processus qui exécutent leur prologue ou épilogue peuvent participer à la décision de laisser entrer ou pas Pi en section critique

Règle 3 : un processus qui demande à entrer en section critique doit pouvoir y entrer au bout d'un temps fini

Exclusion mutuelle : synchronisation par attente active

La réalisation se fait par voie matérielle (hard) : le masquage des interruptions est utilisé :

Principe : Avant d'entrer en SC, le processus masque les interruptions. En sortie de SC, le processus démasque les interruptions

Inconvénients : entrées/sorties en SC (interblocage), SC longue (perte d'interruptions), dangereux un processus peut oublier de démasquer les IT, inadaptée dans le cas multiprocesseur

Synchronisation par attente active et scrutation de variables communes

La réalisation de cette solution se fait par voie logicielle

Principe de la solution : définir un ensemble de variables partagées reflétant l'état d'occupation de la ressource et l'état des processus concurrents pour cette ressource. Avant d'entrer en section critique, un processus exécute un protocole testant et modifiant les variables d'état. Si la ressource n'est pas libre, il exécute une boucle d'attente. En sortant de section critique, un processus "libère" la ressource en modifiant les variables d'état

Tour: contexte commun initialisé à 1

P1	P2
Tant que tour ==2	Tant que tour==1
Fin tant que	Fin tant que
<SC>	<SC>
Tour=2	Tour=1

Avantages

L'exclusion mutuelle est satisfaite. Pour chaque valeur de tour, une section critique et une seule peut s'exécuter, et ce jusqu'à son terme.

L'interblocage est impossible puisque tour prend soit la valeur 1, soit la valeur 2 (Les deux processus ne peuvent pas être bloqués en même temps).

La privation est impossible : un processus ne peut empêcher l'autre d'entrer en section critique puisque tout change de valeur à la fin de chaque section critique.

Inconvénients

P1 et P2 sont contraints de fonctionner avec la même fréquence d'entrée en section critique.

Si l'exécution de P2 s'arrête, celle de P1 s'arrête aussi : le programme est bloqué.